

**Moscow Aviation Institute
(National Research University)**

Faculty of Information Technologies and Applied Mathematics

Department of Computational Mathematics and Programming

Laboratory work No. 3, course “Discrete Analysis”

Student: M. A. Khorokhorin
Teacher: A. A. Kukhtichev
Group: M8O-206BV-24
Date: 10.05.2026
Grade:
Signature:

Moscow, 2026

Laboratory works No. 2 and No. 3: PATRICIA and Quality Research

Time limit	15 seconds
Memory limit	512 MB
Input	standard input or <code>input.txt</code>
Output	standard output or <code>output.txt</code>

Laboratory work No. 2

Task: You need to create a program library implementing the specified data structure, on the basis of which to develop a dictionary program. In the dictionary, each key, which is a case-insensitive sequence of letters of the English alphabet of no more than 256 characters, is associated with a number from 0 to $2^{64} - 1$. Different words can be assigned the same number.

The program must process lines of the input file until its end. Each line can have the following format:

- **+ word 34** — add the word **word** with number 34 to the dictionary. The program must output **OK** if the operation was successful, **Exist** if the word is already in the dictionary.
- **- word** — remove the word **word** from the dictionary. The program must output **OK** if the word existed and was removed, **NoSuchWord** if the word was not found in the dictionary.
- **word** — find the word **word** in the dictionary. The program must output **OK: 34** if the word was found. If the word was not found in the dictionary, output **NoSuchWord**.
- **! Save /path/to/file** — save the dictionary to a file in binary compact representation, at the path specified by the command parameter. If successful, the program must output **OK**; if the operation fails, output a description of the error.
- **! Load /path/to/file** — load the dictionary from a file. It is assumed that the file was previously prepared using the **Save** command. If successful, the program must output **OK** and the loaded dictionary replaces the current one; if unsuccessful, diagnostics must be output and the working dictionary must remain unchanged.

For all operations, if a system error occurs, the program must output a line starting with **ERROR:** and describing the error in English.

The difference between variants lies only in the data structures used.

Data structure variant: PATRICIA.

Laboratory work No. 3: A. Software Quality Research

Task: For the dictionary implementation from the previous laboratory work, it is necessary to investigate execution speed and RAM consumption. If errors or obvious shortcomings are found, they must be fixed.

The result of the work is a report containing:

- a work diary describing what was done and when, which tools were used, and what results were achieved at each step of the laboratory work;
- conclusions about the shortcomings found;
- comparison of the fixed program with the previous version;
- general conclusions about the completed laboratory work and the experience gained.

The minimum set of tools used must include the **gprof** utility and the **dmalloc** library. However, they may be replaced with any similar or more advanced tools, for example Valgrind or Shark, and additional tools may also be used, for example **gcov**.

Additional information

gprof

The **gprof** utility should be studied independently using the operational documentation of the Unix operating system.

dmalloc

Below is a short description of how to use it for a small test program.

Suppose a small program is written with two dynamic memory usage errors that cannot be checked at compile time and can only be detected at run time:

```
#include <stdio.h>
#include <stdlib.h>
```

```

#ifdef DMALLOC
#include "dmalloc.h"
#endif

void *foo() {
    char *p = malloc(1023);
    p[1023] = '\0';
    return p;
}

int main() {
    foo();
    return 0;
}

```

The program has two errors:

1. In line 10, the program accesses the 1024th element of the dynamic array `p`, even though only 1023 elements were allocated. When the program is executed, this error will most likely not be detected, because the actual allocated memory size is usually aligned to the page size and therefore will be larger than 1023 elements.
2. The memory allocated in line 9 is never freed.

If the program is compiled and run, most likely no problems will occur either during compilation or during execution:

```

gcc -Wall dmalloc-test.c -o dtest
./dtest

```

To use the `dmalloc` library, the special header file `dmalloc.h` must be included in the program. It replaces calls to memory allocation functions with its own versions. The `DMALLOC` macro must also be defined to enable the main functionality of the library. In addition, the `DMALLOC_FUNC_CHECK` macro can be defined to enable additional checking of parameters passed to various functions from the C standard library. Finally, the `dmalloc` library must be linked during compilation.

Thus, compilation can be performed as follows:

```

gcc -Wall -DDMALLOC -DDMALLOC_FUNC_CHECK \
    -I/usr/local/include -L/usr/local/lib dmalloc-test.c \
    -o dtest -ldmalloc

```

Before running the program under investigation, the `DMALLOC_OPTIONS` environment variable must be defined. It passes information about how the library should work. The full list of options and their meaning should be found in the library documentation. To correctly set the environment in the `bash` command interpreter, the following command can be used:

```
eval `command dmalloc -b -l logfile -i 100 low`
```

In this case, the check report will be written to the file `logfile`.

Now, if the program is run, it will terminate abnormally:

```
./dtest
```

```
debug-malloc library: dumping program, fatal error
```

```
Error: failed OVER picket-fence magic-number check (err 27)
```

```
Abort trap: 6 (core dumped)
```

Thus, the `dmalloc` library detected an out-of-bounds access. The abnormal program termination also made it possible to create a core dump, which can be examined using the `gdb` debugger.

If the source program is modified by fixing the out-of-bounds access, for example by replacing index 1023 with 1022, the program will run normally. The report will look approximately as follows:

```
1192785155: 1: Dmalloc version '5.5.2' from 'http://dmalloc.com/'
1192785155: 1: flags = 0x4e48503, logfile 'logfile'
1192785155: 1: interval = 100, addr = 0, seen # = 0, limit = 0
1192785155: 1: starting time = 1192785155
1192785155: 1: process pid = 23403
1192785155: 1: Dumping Chunk Statistics:
1192785155: 1: basic-block 8192 bytes, alignment 8 bytes
1192785155: 1: heap address range: 0x16005a000 to 0x160060000, 24576 bytes
1192785155: 1:
1192785155: 1:
1192785155: 1: heap checked 1
1192785155: 1: alloc calls: malloc 1, calloc 0, realloc 0, free 0
1192785155: 1: alloc calls: realloc 0, memalign 0, valloc 0
1192785155: 1: alloc calls: new 0, delete 0
1192785155: 1: user blocks: 1 blocks, 7167 bytes (29%)
1192785155: 1: admin blocks: 2 blocks, 16384 bytes (66%)
1192785155: 1: total blocks: 3 blocks, 24576 bytes
1192785155: 1: current memory in use: 1023 bytes (1 pnts)
1192785155: 1: total memory allocated: 1023 bytes (1 pnts)
1192785155: 1: max in use at one time: 1023 bytes (1 pnts)
1192785155: 1: max allocated with 1 call: 1023 bytes
1192785155: 1: max unused memory space: 1025 bytes (50%)
1192785155: 1: top 10 allocations:
1192785155: 1: total-size count in-use-size count source
```

```

1192785155: 1: 1023 1 1023 1 dmalloc-test.c:9
1192785155: 1: 1023 1 1023 1 Total of 1
1192785155: 1: Dumping Not-Freed Pointers Changed Since Start:
1192785155: 1: not freed: '0x16005b808|s1' (1023 bytes) from 'dmalloc-test.c:9'
1192785155: 1: total-size count source
1192785155: 1: 1023 1 dmalloc-test.c:9
1192785155: 1: 1023 1 Total of 1
1192785155: 1: ending time = 1192785155, elapsed since start = 0:00:00

```

At the end of the report there is a **Dumping Not-Freed Pointers** section, where it can be seen that one memory block of 1023 bytes, allocated in line 9, was not freed before the program finished. Thus, memory leaks in programs can be tracked.

Before performing the laboratory work, it is strongly recommended to independently study the documentation for the **dmalloc** library. Using additional options not described above will improve the impression of the completed work.

In this laboratory work, **dmalloc** is replaced by more advanced tools: Valgrind Memcheck and Valgrind Massif. The research also uses **gprof**, a test generator, and a benchmark that compares PATRICIA with **std::map**.

Input format

Each non-empty line of the input contains one of the commands described above.

Output format

For each input line, the program outputs one response line as specified above.

Example

Input:

```

+ a 1
+ A 2
+ aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa 18446744073709551615
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
A

```

- A

a

Output:

OK

Exist

OK

OK: 18446744073709551615

OK: 1

OK

NoSuchWord

1 Description

The task is to implement a dictionary based on the PATRICIA tree (Practical Algorithm To Retrieve Information Coded In Alphanumeric) and then to study the quality of the resulting program.

A PATRICIA tree is a compressed binary trie in which every node stores the index of the critical bit of the key used for branching. This avoids chains of nodes with a single child and keeps only the positions where the keys actually differ.

The key feature of the structure is *back edges*: when, during a traversal, the critical bit index of the next node is not greater than that of the current node, the traversal stops. The target node of such an edge is the candidate for a full string comparison.

To handle boundary cases correctly, a header node (`header_`) with critical bit zero is used; its left child is the root of the tree. All keys are converted to lower case, which makes the dictionary case-insensitive. The dictionary supports the operations `insert`, `find` and `remove`, as well as binary saving and loading (`saveToFile` and `loadFromFile`).

2 Source code

1 Helper functions

The function `extractBit` extracts the i -th bit of a string (numbering starts at 1, the most significant bit of the first byte is bit 1). The function `firstDifferentBit` finds the position of the first bit in which two keys differ; the result is used during insertion to determine the critical bit of the new node. The function `normalizeKey` converts a key to lower case, and `isValidKey` checks that a key is non-empty, consists of English letters only and is not longer than `MAX_KEY_LENGTH = 256` characters. The functions `writeUnsigned` and `readUnsigned` write and read little-endian integers of a given width; `readUnsigned` throws `PatriciaError` when the file ends too early.

2 Node structure and the tree class

Every node (`PatriciaTrie::Node`) stores a key, a value, the critical bit index and pointers to two children. The field `size_` of the class `PatriciaTrie` tracks the number of elements. The class is movable but not copyable; errors of file operations are reported with the exception class `PatriciaError`.

Method	Purpose
<code>std::optional<std::uint64_t> find(std::string_view key) const</code>	Returns the value of the key or <code>std::nullopt</code>
<code>bool insert(std::string_view key, std::uint64_t value)</code>	Adds a key; returns <code>false</code> if it already exists
<code>bool remove(std::string_view key)</code>	Removes a key; returns <code>false</code> if it is absent
<code>void saveToFile(const std::string& path) const</code>	Writes the dictionary to a binary file
<code>void loadFromFile(const std::string& path)</code>	Replaces the dictionary with the contents of a binary file

3 Search

The function `searchNode` descends the tree, choosing a branch by the critical bit, until it meets a back edge. The node found is only a candidate: `find` additionally performs a full string comparison and returns `std::nullopt` if the keys differ.

4 Insertion

During insertion, `searchNode` is executed first to find a collision or a reference key. Then `firstDifferentBit` computes the critical bit of the new node. After that the tree is descended again to the insertion position, and the new node is attached with a back edge to itself.

5 Removal

Removal handles two cases. If the node being removed has a back edge to itself (`removeNodeWithSelfLoop`), the parent link found by `findParentLink` is switched to the other child. Otherwise (`removeNodeWithoutSelfLoop`) the node that holds the back edge to the removed element is found; its data is moved into the removed node, the links are fixed, and the holder node itself is physically deleted.

6 Serialization

The file starts with the magic header `PATRICIA`, followed by the format version (`FILE_VERSION`, 4 bytes) and the number of entries (8 bytes). The method `saveSubtree` traverses the tree recursively and writes every node as a triple: key length, the key itself and the value. The length is stored as `uint16_t`, because according to the statement a key may be 256 characters long. After writing, the stream state is checked, and on failure `PatriciaError("failed to write file")` is thrown. The method `load` reads the entries into a temporary tree, validates the header, the version, every key and the absence of trailing data; `loadFromFile` replaces the current state with `swap` only after the whole file has been read successfully.

7 Command processing

The class `Dictionary` parses one input line in `Dictionary::execute`: it splits the line into tokens, validates the key and the value and calls the corresponding tree operation. File commands catch `PatriciaError` and print `ERROR:` followed by the error message. The function `runDictionary` reads the input line by line and prints the responses; `main` only sets up fast input and output and calls `runDictionary`.

Listing 1: lab2/include/patricia_trie.hpp

```
1 | #pragma once
2 |
3 | #include <cstdlib>
4 | #include <stdint>
5 | #include <iosfwd>
6 | #include <optional>
```

```

7  #include <stdexcept>
8  #include <string>
9  #include <string_view>
10
11 inline constexpr std::size_t MAX_KEY_LENGTH = 256;
12 inline constexpr std::uint32_t FILE_VERSION = 1;
13
14 class PatriciaError : public std::runtime_error {
15 public:
16     using std::runtime_error::runtime_error;
17 };
18
19 bool isValidKey(std::string_view key);
20 std::string normalizeKey(std::string_view key);
21
22 class PatriciaTrie {
23 public:
24     PatriciaTrie() = default;
25     ~PatriciaTrie();
26
27     PatriciaTrie(const PatriciaTrie&) = delete;
28     PatriciaTrie& operator=(const PatriciaTrie&) = delete;
29     PatriciaTrie(PatriciaTrie&& other) noexcept;
30     PatriciaTrie& operator=(PatriciaTrie&& other) noexcept;
31
32     [[nodiscard]] std::optional<std::uint64_t> find(std::string_view key) const;
33     bool insert(std::string_view key, std::uint64_t value);
34     bool remove(std::string_view key);
35
36     [[nodiscard]] std::size_t size() const { return size_; }
37     [[nodiscard]] bool empty() const { return size_ == 0; }
38     void clear();
39     void swap(PatriciaTrie& other) noexcept;
40
41     void save(std::ostream& output) const;
42     static PatriciaTrie load(std::istream& input);
43     void saveToFile(const std::string& path) const;
44     void loadFromFile(const std::string& path);
45
46 private:
47     struct Node {
48         std::string key;
49         std::uint64_t value = 0;
50         std::size_t checkBit = 0;
51         Node* left = nullptr;
52         Node* right = nullptr;
53     };
54
55     [[nodiscard]] Node* searchNode(std::string_view key) const;
56     [[nodiscard]] Node** findParentLink(const Node* target, std::string_view key) const;
57     void removeNodeWithSelfLoop(Node* target, std::string_view key);
58     void removeNodeWithoutSelfLoop(Node* target, std::string_view key);
59     static void saveSubtree(std::ostream& output, const Node* node);
60     static void destroySubtree(Node* node);
61
62     Node* header_ = nullptr;
63     std::size_t size_ = 0;
64 };

```

Listing 2: lab2/src/patricia_trie.cpp

```

1  #include "patricia_trie.hpp"
2
3  #include <algorithm>
4  #include <array>
5  #include <cstdint>
6  #include <cstring>
7  #include <fstream>
8  #include <istream>
9  #include <optional>
10 #include <ostream>
11 #include <string>
12 #include <string_view>
13 #include <utility>
14
15 namespace {
16
17 constexpr std::size_t BITS_PER_BYTE = 8;
18 constexpr unsigned HIGH_BIT = 0x80U;
19 constexpr unsigned BYTE_MASK = 0xFFU;
20 constexpr std::array<char, 8> FILE_MAGIC = {'P', 'A', 'T', 'R', 'I', 'C', 'I', 'A'};
21
22 bool extractBit(std::string_view key, std::size_t position) {
23     const std::size_t byteIndex = (position - 1) / BITS_PER_BYTE;
24     const std::size_t bitOffset = BITS_PER_BYTE - 1 - (position - 1) % BITS_PER_BYTE;
25     if (byteIndex >= key.size()) {
26         return false;
27     }
28     return ((static_cast<unsigned char>(key[byteIndex]) >> bitOffset) & 1U) != 0;
29 }
30
31 std::size_t firstDifferentBit(std::string_view lhs, std::string_view rhs) {
32     const std::size_t maxLength = std::max(lhs.size(), rhs.size());
33     for (std::size_t i = 0; i < maxLength; ++i) {
34         const unsigned left = i < lhs.size() ? static_cast<unsigned char>(lhs[i]) : 0U;
35         const unsigned right = i < rhs.size() ? static_cast<unsigned char>(rhs[i]) : 0U;
36         if (left != right) {
37             unsigned diff = left ^ right;
38             std::size_t shift = 0;
39             while ((diff & HIGH_BIT) == 0) {
40                 diff <<= 1U;
41                 ++shift;
42             }
43             return i * BITS_PER_BYTE + shift + 1;
44         }
45     }
46     return maxLength * BITS_PER_BYTE + 1;
47 }
48
49 void writeUnsigned(std::ostream& output, std::uint64_t value, std::size_t bytes) {
50     for (std::size_t i = 0; i < bytes; ++i) {
51         output.put(static_cast<char>((value >> (i * BITS_PER_BYTE)) & BYTE_MASK));
52     }
53 }
54
55 std::uint64_t readUnsigned(std::istream& input, std::size_t bytes) {
56     std::uint64_t value = 0;
57     for (std::size_t i = 0; i < bytes; ++i) {
58         const int symbol = input.get();
59         if (symbol == std::char_traits<char>::eof()) {

```

```

60         throw PatriciaError("invalid file format");
61     }
62     value |= static_cast<std::uint64_t>(static_cast<unsigned char>(symbol))
63         << (i * BITS_PER_BYTE);
64 }
65 return value;
66 }
67
68 }
69
70 bool isValidKey(std::string_view key) {
71     return !key.empty() && key.size() <= MAX_KEY_LENGTH &&
72         std::ranges::all_of(key, [](char symbol) {
73             return (symbol >= 'a' && symbol <= 'z') || (symbol >= 'A' && symbol <= 'Z');
74         });
75 }
76
77 std::string normalizeKey(std::string_view key) {
78     std::string result(key);
79     for (char& symbol : result) {
80         if (symbol >= 'A' && symbol <= 'Z') {
81             symbol = static_cast<char>(symbol - 'A' + 'a');
82         }
83     }
84     return result;
85 }
86
87 PatriciaTrie::~PatriciaTrie() {
88     clear();
89 }
90
91 PatriciaTrie::PatriciaTrie(PatriciaTrie&& other) noexcept
92     : header_(std::exchange(other.header_, nullptr)), size_(std::exchange(other.size_, 0)) {
93 }
94
95 PatriciaTrie& PatriciaTrie::operator=(PatriciaTrie&& other) noexcept {
96     if (this != &other) {
97         clear();
98         header_ = std::exchange(other.header_, nullptr);
99         size_ = std::exchange(other.size_, 0);
100     }
101     return *this;
102 }
103
104 void PatriciaTrie::clear() {
105     if (header_ != nullptr) {
106         destroySubtree(header_);
107     }
108     header_ = nullptr;
109     size_ = 0;
110 }
111
112 void PatriciaTrie::swap(PatriciaTrie& other) noexcept {
113     std::swap(header_, other.header_);
114     std::swap(size_, other.size_);
115 }
116
117 PatriciaTrie::Node* PatriciaTrie::searchNode(std::string_view key) const {
118     const Node* previous = header_;
119     Node* current = header_>left;
120     while (current->checkBit > previous->checkBit) {
121         previous = current;

```

```

122     current = extractBit(key, current->checkBit) ? current->right : current->left;
123 }
124 return current;
125 }
126
127 std::optional<std::uint64_t> PatriciaTrie::find(std::string_view key) const {
128     if (header_ == nullptr) {
129         return std::nullopt;
130     }
131     const Node* found = searchNode(key);
132     if (found->key != key) {
133         return std::nullopt;
134     }
135     return found->value;
136 }
137
138 bool PatriciaTrie::insert(std::string_view key, std::uint64_t value) {
139     if (header_ == nullptr) {
140         header_ = new Node{.key = std::string(key), .value = value};
141         header_->left = header_;
142         size_ = 1;
143         return true;
144     }
145
146     const Node* existing = searchNode(key);
147     if (existing->key == key) {
148         return false;
149     }
150
151     const std::size_t diffBit = firstDifferentBit(key, existing->key);
152
153     Node* previous = header_;
154     Node* current = header_->left;
155     while (current->checkBit > previous->checkBit && current->checkBit < diffBit) {
156         previous = current;
157         current = extractBit(key, current->checkBit) ? current->right : current->left;
158     }
159
160     Node* node = new Node{.key = std::string(key), .value = value, .checkBit = diffBit};
161     if (extractBit(key, diffBit)) {
162         node->right = node;
163         node->left = current;
164     } else {
165         node->left = node;
166         node->right = current;
167     }
168
169     if (previous == header_) {
170         header_->left = node;
171     } else if (extractBit(key, previous->checkBit)) {
172         previous->right = node;
173     } else {
174         previous->left = node;
175     }
176
177     ++size_;
178     return true;
179 }
180
181 PatriciaTrie::Node** PatriciaTrie::findParentLink(const Node* target, std::string_view key) const {
182     Node** link = &header_->left;
183     const Node* parent = header_;

```

```

184     Node* current = header_>left;
185     while (current->checkBit > parent->checkBit && current != target) {
186         parent = current;
187         link = extractBit(key, current->checkBit) ? &current->right : &current->left;
188         current = *link;
189     }
190     return link;
191 }
192
193 void PatriciaTrie::removeNodeWithSelfLoop(Node* target, std::string_view key) {
194     if (target == header_) {
195         delete header_;
196         header_ = nullptr;
197         return;
198     }
199     Node* other = target->left == target ? target->right : target->left;
200     *findParentLink(target, key) = other;
201     delete target;
202 }
203
204 void PatriciaTrie::removeNodeWithoutSelfLoop(Node* target, std::string_view key) {
205     Node* grandParent = header_;
206     Node* holder = header_;
207     Node* current = header_>left;
208     while (current->checkBit > holder->checkBit) {
209         grandParent = holder;
210         holder = current;
211         current = extractBit(key, current->checkBit) ? current->right : current->left;
212     }
213
214     const bool holderHasSelfLoop = holder->left == holder || holder->right == holder;
215     Node* holderChild = nullptr;
216     if (holderHasSelfLoop) {
217         holderChild = target;
218     } else {
219         holderChild = holder->left == target ? holder->right : holder->left;
220     }
221
222     Node* source = header_;
223     current = header_>left;
224     while (current->checkBit > source->checkBit) {
225         source = current;
226         current = extractBit(holder->key, current->checkBit) ? current->right : current->left;
227     }
228
229     target->key = std::move(holder->key);
230     target->value = holder->value;
231
232     if (source->left == holder) {
233         source->left = target;
234     }
235     if (source->right == holder) {
236         source->right = target;
237     }
238     if (target->left == holder) {
239         target->left = holderChild;
240     }
241     if (target->right == holder) {
242         target->right = holderChild;
243     }
244     if (grandParent->left == holder) {
245         grandParent->left = holderChild;

```

```

246     } else if (grandParent->right == holder) {
247         grandParent->right = holderChild;
248     }
249
250     delete holder;
251 }
252
253 bool PatriciaTrie::remove(std::string_view key) {
254     if (header_ == nullptr) {
255         return false;
256     }
257     Node* target = searchNode(key);
258     if (target->key != key) {
259         return false;
260     }
261     if (target->left == target || target->right == target) {
262         removeNodeWithSelfLoop(target, key);
263     } else {
264         removeNodeWithoutSelfLoop(target, key);
265     }
266     --size_;
267     return true;
268 }
269
270 void PatriciaTrie::saveSubtree(std::ostream& output, const Node* node) {
271     writeUnsigned(output, node->key.size(), sizeof(std::uint16_t));
272     output.write(node->key.data(), static_cast<std::streamsize>(node->key.size()));
273     writeUnsigned(output, node->value, sizeof(std::uint64_t));
274     if (node->left != nullptr && node->left->checkBit > node->checkBit) {
275         saveSubtree(output, node->left);
276     }
277     if (node->right != nullptr && node->right->checkBit > node->checkBit) {
278         saveSubtree(output, node->right);
279     }
280 }
281
282 void PatriciaTrie::destroySubtree(Node* node) {
283     if (node->left != nullptr && node->left->checkBit > node->checkBit) {
284         destroySubtree(node->left);
285     }
286     if (node->right != nullptr && node->right->checkBit > node->checkBit) {
287         destroySubtree(node->right);
288     }
289     delete node;
290 }
291
292 void PatriciaTrie::save(std::ostream& output) const {
293     output.write(FILE_MAGIC.data(), static_cast<std::streamsize>(FILE_MAGIC.size()));
294     writeUnsigned(output, FILE_VERSION, sizeof(std::uint32_t));
295     writeUnsigned(output, size_, sizeof(std::uint64_t));
296     if (header_ != nullptr) {
297         saveSubtree(output, header_);
298     }
299     output.flush();
300     if (!output.good()) {
301         throw PatriciaError("failed to write file");
302     }
303 }
304
305 PatriciaTrie PatriciaTrie::load(std::istream& input) {
306     std::array<char, FILE_MAGIC.size()> magic{};
307     if (!input.read(magic.data(), static_cast<std::streamsize>(magic.size()))) ||

```

```

308     magic != FILE_MAGIC) {
309         throw PatriciaError("invalid file format");
310     }
311     if (readUnsigned(input, sizeof(std::uint32_t)) != FILE_VERSION) {
312         throw PatriciaError("unsupported file version");
313     }
314     const std::uint64_t count = readUnsigned(input, sizeof(std::uint64_t));
315
316     PatriciaTrie trie;
317     for (std::uint64_t i = 0; i < count; ++i) {
318         const std::uint64_t length = readUnsigned(input, sizeof(std::uint16_t));
319         if (length == 0 || length > MAX_KEY_LENGTH) {
320             throw PatriciaError("invalid file format");
321         }
322         std::string key(length, '\0');
323         if (!input.read(key.data(), static_cast<std::streamsize>(length)) || !isValidKey(key) ||
324             normalizeKey(key) != key) {
325             throw PatriciaError("invalid file format");
326         }
327         const std::uint64_t value = readUnsigned(input, sizeof(std::uint64_t));
328         if (!trie.insert(key, value)) {
329             throw PatriciaError("invalid file format");
330         }
331     }
332     if (input.peek() != std::char_traits<char>::eof()) {
333         throw PatriciaError("invalid file format");
334     }
335     return trie;
336 }
337
338 void PatriciaTrie::saveToFile(const std::string& path) const {
339     std::ofstream file(path, std::ios::binary | std::ios::trunc);
340     if (!file) {
341         throw PatriciaError("cannot open file for writing");
342     }
343     save(file);
344     file.close();
345     if (!file) {
346         throw PatriciaError("failed to write file");
347     }
348 }
349
350 void PatriciaTrie::loadFromFile(const std::string& path) {
351     std::ifstream file(path, std::ios::binary);
352     if (!file) {
353         throw PatriciaError("cannot open file for reading");
354     }
355     PatriciaTrie loaded = load(file);
356     swap(loaded);
357 }

```

Listing 3: lab2/include/dictionary.hpp

```

1  #pragma once
2
3  #include <iosfwd>
4  #include <optional>
5  #include <string>
6  #include <string_view>
7
8  #include "patricia_trie.hpp"

```

```

9
10 class Dictionary {
11 public:
12     std::optional<std::string> execute(std::string_view line);
13
14 private:
15     std::string add(std::string_view key, std::string_view value);
16     std::string erase(std::string_view key);
17     std::string lookup(std::string_view key) const;
18     std::string runFileCommand(std::string_view command, const std::string& path);
19
20     PatriciaTrie trie_;
21 };
22
23 void runDictionary(std::istream& input, std::ostream& output);

```

Listing 4: lab2/src/dictionary.cpp

```

1 #include "dictionary.hpp"
2
3 #include <algorithm>
4 #include <charconv>
5 #include <cstdint>
6 #include <cstdint>
7 #include <istream>
8 #include <new>
9 #include <optional>
10 #include <ostream>
11 #include <string>
12 #include <string_view>
13 #include <system_error>
14 #include <vector>
15
16 #include "patricia_trie.hpp"
17
18 namespace {
19
20 bool isSpace(char symbol) {
21     return symbol == ' ' || symbol == '\t' || symbol == '\r' || symbol == '\n' || symbol == '\v' ||
22            symbol == '\f';
23 }
24
25 std::vector<std::string_view> splitTokens(std::string_view line) {
26     std::vector<std::string_view> tokens;
27     std::size_t position = 0;
28     while (position < line.size()) {
29         while (position < line.size() && isSpace(line[position])) {
30             ++position;
31         }
32         const std::size_t start = position;
33         while (position < line.size() && !isSpace(line[position])) {
34             ++position;
35         }
36         if (position > start) {
37             tokens.push_back(line.substr(start, position - start));
38         }
39     }
40     return tokens;
41 }
42
43 std::optional<std::uint64_t> parseValue(std::string_view text) {

```

```

44     const bool allDigits =
45         std::ranges::all_of(text, [](char symbol) { return symbol >= '0' && symbol <= '9'; });
46     if (text.empty() || !allDigits) {
47         return std::nullopt;
48     }
49     std::uint64_t value = 0;
50     const auto [end, error] = std::from_chars(text.data(), text.data() + text.size(), value);
51     if (error != std::errc{} || end != text.data() + text.size()) {
52         return std::nullopt;
53     }
54     return value;
55 }
56
57 constexpr std::string_view INVALID_COMMAND = "ERROR: invalid command";
58 constexpr std::string_view UNKNOWN_COMMAND = "ERROR: unknown command";
59 constexpr std::string_view INVALID_KEY = "ERROR: invalid key";
60 constexpr std::string_view INVALID_VALUE = "ERROR: invalid value";
61
62 }
63
64 std::optional<std::string> Dictionary::execute(std::string_view line) {
65     const std::vector<std::string_view> tokens = splitTokens(line);
66     if (tokens.empty()) {
67         return std::nullopt;
68     }
69     const std::string_view command = tokens.front();
70     if (command == "+") {
71         if (tokens.size() != 3) {
72             return std::string(INVALID_COMMAND);
73         }
74         return add(tokens[1], tokens[2]);
75     }
76     if (command == "-") {
77         if (tokens.size() != 2) {
78             return std::string(INVALID_COMMAND);
79         }
80         return erase(tokens[1]);
81     }
82     if (command == "!") {
83         if (tokens.size() < 2 || (tokens[1] != "Save" && tokens[1] != "Load")) {
84             return std::string(UNKNOWN_COMMAND);
85         }
86         if (tokens.size() != 3) {
87             return std::string(INVALID_COMMAND);
88         }
89         return runFileCommand(tokens[1], std::string(tokens[2]));
90     }
91     if (tokens.size() != 1) {
92         return std::string(UNKNOWN_COMMAND);
93     }
94     return lookup(command);
95 }
96
97 std::string Dictionary::add(std::string_view key, std::string_view value) {
98     if (!isValidKey(key)) {
99         return std::string(INVALID_KEY);
100     }
101     const std::optional<std::uint64_t> number = parseValue(value);
102     if (!number) {
103         return std::string(INVALID_VALUE);
104     }
105     return trie_.insert(normalizeKey(key), *number) ? "OK" : "Exist";

```

```

106 }
107
108 std::string Dictionary::erase(std::string_view key) {
109     if (!isValidKey(key)) {
110         return std::string(INVALID_KEY);
111     }
112     return trie_.remove(normalizeKey(key)) ? "OK" : "NoSuchWord";
113 }
114
115 std::string Dictionary::lookup(std::string_view key) const {
116     if (!isValidKey(key)) {
117         return std::string(INVALID_KEY);
118     }
119     const std::optional<std::uint64_t> value = trie_.find(normalizeKey(key));
120     if (!value) {
121         return "NoSuchWord";
122     }
123     return "OK: " + std::to_string(*value);
124 }
125
126 std::string Dictionary::runFileCommand(std::string_view command, const std::string& path) {
127     try {
128         if (command == "Save") {
129             trie_.saveToFile(path);
130         } else {
131             trie_.loadFromFile(path);
132         }
133     } catch (const PatriciaError& error) {
134         return std::string("ERROR: ") + error.what();
135     }
136     return "OK";
137 }
138
139 void runDictionary(std::istream& input, std::ostream& output) {
140     Dictionary dictionary;
141     std::string line;
142     while (std::getline(input, line)) {
143         std::optional<std::string> response;
144         try {
145             response = dictionary.execute(line);
146         } catch (const std::bad_alloc&) {
147             response = "ERROR: out of memory";
148         }
149         if (response) {
150             output << *response << '\n';
151         }
152     }
153 }

```

Listing 5: lab2/app/main.cpp

```
1 | #include <iostream>
2 |
3 | #include "dictionary.hpp"
4 |
5 | int main() {
6 |     std::ios::sync_with_stdio(false);
7 |     std::cin.tie(nullptr);
8 |
9 |     runDictionary(std::cin, std::cout);
10 |    return 0;
11 | }
```

3 Console

```
root@container:/workspace# make build PRESET=release
root@container:/workspace# ./build/release/lab3/lab3_main
+ a 1
OK
+ A 2
Exist
A
OK: 1
- A
OK
a
NoSuchWord
```

Commands used for the research:

```
root@container:/workspace# python3 lab3/benchmark/generator.py 100000 16 42
root@container:/workspace# build/release/lab3/lab3_profile \
    < lab3/benchmark/tests.txt
root@container:/workspace# gprof build/release/lab3/lab3_profile gmon.out \
    > gprof.txt
root@container:/workspace# valgrind --leak-check=full \
    build/release/lab3/lab3_main < lab3/benchmark/tests.txt
root@container:/workspace# valgrind --tool=massif \
    --massif-out-file=massif.out \
    build/release/lab3/lab3_main < lab3/benchmark/tests.txt
```

4 Performance research results

This section presents the results of testing the PATRICIA tree compared with the standard container `std::map`, as well as the analysis of memory consumption and the profiling of the fixed program.

1 RAM consumption analysis

The Valgrind Memcheck tool was used to check for dynamic memory errors. The run was performed on a test set of 100 000 insertions, 100 000 lookups and 100 000 removals.

HEAP SUMMARY:

```
in use at exit: 122,880 bytes in 6 blocks
total heap usage: 729,452 allocs, 729,446 frees, 24,697,187 bytes allocated
```

LEAK SUMMARY:

```
definitely lost: 0 bytes in 0 blocks
indirectly lost: 0 bytes in 0 blocks
possibly lost: 0 bytes in 0 blocks
still reachable: 122,880 bytes in 6 blocks
```

ERROR SUMMARY: 0 errors from 0 contexts

Metric	PATRICIA	Comment
Memory errors	0	No invalid reads or writes were found
User memory leaks	0 bytes	Tree nodes are freed correctly
still reachable	122 880 bytes	Buffers of the standard I/O library
Heap peak according to Massif	7.335 MB	Set of 100 000 words

Memory conclusion: no leaks were found in the user code. The memory reported as **still reachable** belongs to internal libstdc++ buffers and is not an error of the dictionary implementation.

2 Execution speed research

Time was measured for insertion, lookup and removal of string keys. The comparison was made with `std::map`.

	PATRICIA, μs			std::map, μs		
N	Insert	Find	Remove	Insert	Find	Remove
1 000	348	208	465	214	161	303
10 000	4 731	3 246	8 264	4 251	2 405	3 711
100 000	86 469	75 838	100 506	87 293	73 691	81 991
500 000	731 836	695 895	764 450	766 442	748 168	785 446
1 000 000	1 842 835	2 022 130	2 038 648	1 953 836	1 985 049	2 322 171

On small data sets `std::map` is faster because of its lower overhead. On large sets PATRICIA starts to win: for $N = 500\,000$ all three operations are faster, and for $N = 1\,000\,000$ insertion and removal are faster.

3 Performance analysis with gprof

For a detailed analysis of the execution time, the target `lab3_profile` was built with the `-pg` flag. The `gprof` report for 100 000 operations of each type showed the following main lines (`make research` on the current version of the code).

Function	Share of time, %	Number of calls
<code>main</code>	59.20	—
<code>PatriciaTrie::remove</code>	16.33	100 000
<code>PatriciaTrie::find</code>	12.25	100 000
<code>PatriciaTrie::insert</code>	10.21	100 000
<code>PatriciaTrie::removeNodeWithoutSelfLoop</code>	2.04	68 956
<code>PatriciaTrie::removeNodeWithSelfLoop</code>	0.00	31 044
<code>normalizeKey</code>	0.00	300 000

Profiling shows that most of the time (59%) is spent in `main`, which reads and parses the 300 000 input commands and runs the `std::map` part of the benchmark; the tree operations take the rest. Removal is the most expensive PATRICIA operation, because after the physical removal of a node the back edges have to be restored: about 69% of the removals hit a node without a self loop and need the more complex `removeNodeWithoutSelfLoop`.

4 Comparison of the original and the fixed version

The fixes did not change the asymptotic complexity of the tree operations: search, insertion and removal still descend the tree by critical bits in the same way. The main benefit of the fixes is correct handling of serialization errors and invalid input, and a fair benchmark in which the lookup loop cannot be removed by the optimizer.

5 Work diary

- Stage 1: Study of the PATRICIA tree. Analysis of critical bits, back edges and the header node.
- Stage 2: Implementation of the dictionary. The operations `insert`, `find`, `remove`, case-insensitive key handling and binary serialization were added.
- Stage 3: Preparation of the benchmark and the test generator (`lab3/benchmark/generator.py`). PATRICIA was compared with `std::map` on sets of 1 000 to 1 000 000 keys.
- Stage 4: Quality research. `gprof`, Valgrind Memcheck and Valgrind Massif were used; data on execution time and memory were obtained.
- Stage 5: Fixing the defects. Errors in file loading, binary format checking, saving a key of length 256 and the lookup measurement in the benchmark were fixed.
- Stage 6: Rebuilding the report and a final check that the results are up to date.

6 Defects found and fixes

During the research the following problems were found and fixed:

1. Loading a missing file: the load operation reported success when the file could not be opened, and all load failures were reported with one generic message. Fixed: file operations throw `PatriciaError`, and the program prints a specific message, for example `ERROR: cannot open file for reading`, `ERROR: invalid file format` or `ERROR: unsupported file version`.
2. The binary file had no header, so an arbitrary or foreign file could be accepted. Fixed: the file starts with the magic header `PATRICIA`, followed by the format version and the number of entries; all of them are checked during loading.
3. A damaged binary file could be accepted partially. Fixed: loading is performed into a temporary tree, the completeness and validity of every entry and the absence of trailing data are checked, and the working dictionary is replaced only after the whole file has been read successfully.
4. The key length was stored as `uint8_t`, which overflowed for a key of 256 characters. Fixed: the length is stored as `uint16_t`.
5. Write errors were not reported reliably. Fixed: the stream state is checked after writing, and on failure the program prints `ERROR: failed to write file`.
6. Keys were not validated. Fixed: a key must consist of English letters only and be no longer than 256 characters; otherwise `ERROR: invalid key` is printed.
7. Unknown commands starting with `!` were silently ignored. Fixed: the program prints `ERROR: unknown command`.
8. The benchmark did not use the result of `PATRICIA` lookups, so the compiler could remove the measured loop. Fixed: the benchmark counts successful lookups and prints the counter, so the loop cannot be optimized away.

7 Conclusions

During the laboratory works, the PATRICIA data structure was implemented for a dictionary of string keys, and the quality of the resulting program was studied. Lookup, insertion, removal, saving and loading of the dictionary from a binary file were implemented.

Profiling with `gprof` showed that, apart from reading and parsing the input, most of the time is spent in the removal, lookup and insertion operations, which matches the structure of the program: these functions traverse the tree by critical bits.

Valgrind Memcheck found no memory leaks and no invalid accesses in the dictionary code. Massif showed moderate memory consumption: about 7.335 MB of heap memory for a set of 100 000 words.

Functional defects of serialization were fixed: loading a missing file, a damaged file or a file without the correct header, and saving a key of the maximum length of 256 characters. Error messages became specific, keys are validated, and unknown commands are reported. The benchmark was also fixed so that the PATRICIA lookup measurement cannot be reduced to an empty loop by the optimizer.

The main result of the work is that the program became more reliable on erroneous input and is measured more fairly in the performance tests, while the underlying data structure and its asymptotic properties remained the same.

References

- [1] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. *Introduction to Algorithms, 2nd edition*. — MIT Press, 2001.
- [2] Morrison D. R. *PATRICIA — Practical Algorithm To Retrieve Information Coded In Alphanumeric*. Journal of the ACM, 15(4):514–534, 1968.
- [3] *GNU gprof*.
URL: <https://sourceware.org/binutils/docs/gprof/> (accessed: 10.05.2026).
- [4] *Valgrind User Manual*.
URL: <https://valgrind.org/docs/manual/> (accessed: 10.05.2026).